CPU and Memory Stress Testing Tool: A Web-Based Operating System Performance Testing and Monitoring Application

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

V. VEDAPRIYA	**2520030376**
B. SRISAIHARINI	**2520030384**
K. SAI SAMYUKTHA	**2520030443**

Under the Esteemed Guidance of

Dr. HARIKA
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"CPU and Memory Stress Testing Tool"** is a record of Bonafide work of **VUNNAM VEDA PRIYA-2520030376, BRUNDAVANAM VR VR SRISAIHARINI - 2520030384, KANAPARTHY SAI SAMYUKTHA - 2520030443** submitted in
partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

V.VEDAPRIYA	**2520030376**
K. SAI SAMYUKTHA	**2520030443**
B. SRISAIHARINI	**2520030384**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled**"CPU AND MEMORY STRESS TESTING TOOL"** is a bonafide work done and submitted by**V.VedaPriya– 2520030376 ,K.Sai Samyuktha– 2520030443, B.SriSaiHarini – 2520030384**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide Dr.Harika**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. We also acknowledge the open-source technologies used in the implementation, including Python, FastAPI, psutil, React, Vite, Recharts, and the testing libraries that support the application. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

V.VEDAPRIYA - 2520030376

B.VR VR SRISAIHARINI - 2520030384

K. SAI SAMYUKTHA - 2520030443

Abstract

CPU and Memory Stress Testing Tool is a browser-based systems engineering application designed to generate controlled CPU and memory workloads while monitoring the performance of the host computer. It combines a Python backend built with FastAPI, multiprocessing-based workload engines, psutil-based system telemetry, and a React frontend. Users can configure CPU-only, memory-only, or combined tests, set workload parameters and duration, observe live CPU and RAM measurements, stop a test when necessary, and review historical results.

The application collects system information such as processor details, physical and logical core counts, total and available memory, CPU utilization, per-core utilization, memory usage, swap usage, CPU frequency when available, and system uptime. Live metrics are delivered to the frontend through a **WebSocket** endpoint. Test reports are stored locally as **JSON** and can be exported as JSON or CSV for further analysis. Safety checks constrain the test duration and the worker count and preserve a configurable memory reserve to reduce the risk of exhausting the system memory. The project demonstrates operating-system concepts including process management, CPU scheduling, memory allocation, demand paging, inter-process coordination, workload cancellation, resource monitoring, and performance reporting.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–11
		1.1 Background of the Project	8
		1.2 Problem Statement	8
		1.3 Scope of the Project	9
		1.4 Objectives of the Project	10
		1.5 Importance of the Application	11
		1.6 Target Users / Audience	11
2	OS Concepts and Study		12–17
		2.1 Operating System Concepts Related to the Project	12
		2.2 Existing Methods / Algorithms	13
		2.3 Proposed Approach	14
		2.4 Advantages of the Proposed Approach	16
3	System Requirements		18–20
		3.1 Hardware Requirements	18
		3.2 Software Requirements	18
		3.3 Operating System Environment – Windows 10/11	19
		3.4 Development Tools and Libraries	19
		3.5 Constraints	20
4	Technology and Tools Used		21–23
		4.1 Python, FastAPI and Uvicorn	21
		4.2 psutil	21
		4.3 Python Multiprocessing	21
		4.4 WebSockets and Data Persistence	22
		4.5 React, Vite and Recharts	22
		4.6 Main Project Structure	22
5	System Architecture and Design		24–27
		5.1 Overall System Architecture	24
		5.2 System Workflow	24
		5.3 Flowchart / Data Flow	25
		5.4 Module Description	26
		5.5 Input and Output Design	27
6	Implementation		28–33
		6.1 Module-wise Implementation	28
		6.2 Algorithm / Pseudocode	29
		6.3 Core Python Implementation	30

S. No.	Chapter	Topics / Subtopics	Page No.
		6.4 System Calls / APIs Used	31
		6.5 Process / Thread Implementation	31
		6.6 IPC / Synchronization Implementation	32
		6.7 Memory Management Implementation	32
		6.8 User Interface Implementation	32
		6.9 Report Generation and Persistence	33
7	Execution and Demonstration		34–39
		7.1 Installation and Setup	34
		7.2 Execution Commands	34
		7.3 Sample Inputs and Outputs	34
		7.4 API Reference and Testing Strategy	35
		7.5 Screenshots of Execution	36
		7.6 Demonstration Procedure	38
		7.7 Demonstration of OS Concepts	38
8	Conclusion		40–42
		8.1 Summary of the Project	40
		8.2 Objectives Achieved	40
		8.3 Results Achieved	41
		8.4 Skills and Learning Outcomes	41
		8.5 Limitations	41
		8.6 Future Enhancements	42
9	References		43–44
		9.1 Books	43
		9.2 Research Papers	43
		9.3 Python and Library Documentation	43
		9.4 Online Resources	43
	Appendix A - Sample Test Configuration Checklist		45
	Appendix B - Report Fields		46

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

A computer system runs many processes that compete for processor time, physical memory, and other resources. An Operating System is responsible for sharing these resources among the running programs. Understanding how a system behaves under a controlled workload is therefore useful when evaluating performance, checking resource limits, and studying **operating-system behaviour**. **Stress testing** deliberately increases resource demand so that measurements can be observed while the system is under load.

The **CPU and Memory Stress Testing Tool** is a browser-based systems engineering application that provides a graphical way to create repeatable CPU and memory workloads. Instead of relying only on terminal commands and separate monitoring utilities, a user can configure a test, observe live system telemetry, stop a running test, and compare saved reports from one web interface.

The application combines a **Python backend** built with FastAPI, **multiprocessing-based workload engines**, **psutil-based system telemetry**, and a **React frontend**. The backend generates the workload and collects measurements from the host computer, while the frontend presents the results through dashboard cards and charts.

1.2 Problem Statement

Manual performance testing often requires several commands and separate monitoring tools. It can be difficult to configure a repeatable workload, monitor CPU and memory changes at the same time, safely stop a test, and preserve the results for later comparison. Uncontrolled stress workloads can also consume too many resources and make a computer unresponsive.

Therefore, the proposed system addresses the need for a centralized web-based tool that can generate controlled workloads and measure their effect on the host system. The tool is designed to:

- Generate controlled CPU-only, memory-only, and combined stress workloads.
- Provide a centralized dashboard for configuring and running tests.
- Monitor CPU, per-core CPU, memory, and swap usage in real time.

- Stop a running test safely through manual cancellation or automatic completion.
- Apply safety limits to test duration, worker count, and memory consumption.
- Preserve test results as reports that can be reviewed, compared, and exported.

The project addresses these issues through a centralized dashboard, configurable workloads, live telemetry, report history, export options, and safety limits.

1.3 Scope of the Project

The supplied implementation targets a **Windows desktop environment** and is accessed through a modern web browser. The backend exposes REST endpoints and a WebSocket stream, and the frontend is a React single-page application. Test reports are stored as local JSON files under the project's data/reports directory. The project is intended for controlled local testing and educational demonstrations, not as a replacement for a production-grade enterprise monitoring platform.

The system includes the following major functionalities:

- **System Information:** Displays processor details, physical and logical core counts, total memory, operating-system details, and system uptime.
- **CPU Monitoring:** Displays total CPU utilization and per-core CPU utilization, along with CPU frequency when it is available.
- **Memory Monitoring:** Displays total, used, and available RAM, RAM percentage, and swap usage.
- **CPU Stress Testing:** Runs CPU-bound work in multiple worker processes with a configurable intensity.
- **Memory Stress Testing:** Allocates memory in bounded chunks and touches the allocated pages while preserving a safety reserve.
- **Combined Testing:** Runs the CPU and memory workloads together for the configured duration.
- **Test Control:** Allows the user to start a test, observe its progress, and abort it at any time.
- **Live Telemetry:** Streams metric snapshots to the dashboard through a WebSocket endpoint.
- **Report Generation:** Saves test summaries and time-series metrics locally as

JSON reports.

- **Report Export and Comparison:** Supports JSON and CSV export and the comparison of selected runs.
- **Settings:** Allows safety limits and warning thresholds to be viewed, updated, and reset to defaults.

The supplied design is intended for local desktop-browser use. Authentication, role-based access control, and remote multi-machine monitoring are not part of the current implementation.

1.4 Objectives of the Project

The main objective of the project is to develop a simple and centralized web application that can generate controlled CPU and memory workloads and observe the resulting system behaviour. The specific objectives are:

- To generate controlled CPU-only, memory-only, and combined stress workloads.
- To monitor total CPU utilization and per-core CPU utilization.
- To display memory, swap, processor, operating-system, and uptime information.
- To allow users to configure test duration, worker count, CPU intensity, memory target, and monitoring interval within defined limits.
- To provide live telemetry through WebSockets and display the metrics using dashboard charts.
- To support manual cancellation and automatic completion when the configured duration ends.
- To preserve a memory safety reserve and stop memory allocation when the available memory becomes too low.
- To save test summaries and time-series metrics locally and support JSON and CSV export.
- To demonstrate Operating System concepts including multiprocessing, CPU scheduling, virtual memory, process lifecycle, and resource management.

1.5 Importance of the Application

Resource saturation is one of the most common causes of poor application performance. Observing how a system responds when the processor is fully loaded or when free memory becomes scarce helps users understand the limits of their computers and the behaviour of the Operating System under pressure.

The application can be used in Operating Systems laboratories, introductory performance engineering exercises, application benchmarking experiments, and demonstrations of resource saturation and recovery. It helps users connect measured system behaviour with **CPU scheduling, multiprocessing, memory commitment, and process cleanup**.

The reports generated by the tool make it possible to compare runs with different workload settings. Because the workload is configurable and the results are saved, experiments become more consistent and repeatable than manual testing with separate utilities.

Therefore, the project connects theoretical Operating System concepts with practical measurements obtained from a real running system.

1.6 Target Users / Audience

The proposed application can be useful for different categories of users.

1. Students

Students studying Operating Systems and system programming can use the tool to observe how processes, CPU scheduling, and memory allocation affect the measured behaviour of a real computer.

2. Developers

Developers can use the tool to check how their applications behave when CPU or memory resources are constrained by running a controlled workload in the background.

3. Lab Instructors

Lab instructors can use the application to demonstrate resource utilization and system monitoring in a classroom or laboratory session using a single dashboard.

4. Technical Users

Technical users can perform short, controlled local performance experiments and keep the exported reports for later comparison.

CHAPTER 2 – OS CONCEPTS AND STUDY

2.1 Operating System Concepts Related to the Project

The CPU and Memory Stress Testing Tool is based on several important concepts of Operating Systems. Since the project generates workloads and measures their effect on the host computer, concepts such as process management, multiprocessing, CPU scheduling, memory management, demand paging, inter-process communication, process lifecycle, and resource monitoring are directly related to the implementation.

Process Management and Multiprocessing

A process is an instance of a program being executed by the Operating System. Each process has its own execution context and its own virtual address space. The CPU stress engine uses Python's multiprocessing facilities to create worker processes. Separate processes can execute CPU-bound work across multiple logical processors without relying on Python threads alone for parallel execution.

CPU Scheduling and Workload Intensity

The Operating System scheduler shares processor time among runnable processes. The CPU workload engine varies its effective intensity using a **duty-cycle pattern**: workers perform computational work during a busy portion of a cycle and yield by sleeping during the remaining portion. This makes it possible to compare different configured intensities and observe their effect on CPU utilization.

Memory Management and Demand Paging

Modern Operating Systems distinguish the reservation of virtual address space from the physical memory that is actually committed to a process's working set. The memory engine allocates memory in bounded chunks and writes data into the allocated buffers so that the pages are touched. It monitors the available memory and preserves a configurable reserve for the Operating System and other applications. The safety mechanism is designed to stop further allocation when the reserve threshold is reached.

Inter-Process Communication (IPC)

The workload engines use multiprocessing coordination primitives, including **events** and **shared values**, to communicate stop requests and to report the allocated memory or worker status. These primitives allow the test manager and the worker processes to

coordinate without sharing ordinary program variables.

Process Lifecycle and Cancellation

The test manager coordinates the lifecycle of a test. The states handled by the system include:

- IDLE – no test is active and the system is ready to accept a new request.
- STARTING – a request has been validated and the workers are being created.
- RUNNING – the workload is active and metrics are being sampled.
- STOPPING – a stop request has been received and cleanup is in progress.
- COMPLETED – the configured duration ended and the report was saved.
- STOPPED – the test was cancelled by the user.
- FAILED – an error occurred during the test.

A user can request cancellation, and the manager attempts to stop the workers and clean up the resources. When a worker process exits, the Operating System reclaims its process resources.

Resource Monitoring

Resource monitoring is the measurement of CPU utilization, per-core utilization, memory usage, swap usage, CPU frequency, and uptime. The application samples these values at a configurable interval and uses warning thresholds for CPU and RAM so that high utilization can be identified easily.

2.2 Existing Methods / Algorithms

Several existing tools and methods are used to observe and stress a computer system. However, each has limitations when used for a configurable and repeatable classroom experiment.

Existing approach	Limitation for a classroom experiment	How this project helps
Task Manager / system monitor	Useful for observation, but not a complete configurable test-and-report workflow.	Combines workload configuration, telemetry, and saved reports.
Command-line stress utilities	May require separate setup and manual result collection.	Provides browser-based controls and report exports.

Existing approach	Limitation for a classroom experiment	How this project helps
One-time metric snapshots	Do not show how metrics change throughout a test.	Collects time-series samples during a run.
Manual spreadsheet recording	Can be inconsistent and time-consuming.	Persists report data and provides JSON/CSV exports.

Limitations of Existing Methods

Although these tools are useful, users generally need to combine several of them to configure a workload, observe the measurements, stop the workload safely, and record the results. This makes experiments difficult to repeat and compare. The proposed system addresses these limitations by combining workload generation, live monitoring, safety limits, and report storage in a single application.

Basic Stress Testing Algorithm

The main operation of the application can be described as follows:

- Start the backend and the frontend of the application.
- Retrieve the system information and the current metrics.
- Accept the test type and the workload parameters from the user.
- Validate the duration, the worker count, and the memory safety budget.
- Start the CPU workers and/or the memory worker.
- Sample the system metrics periodically and stream them to the dashboard.
- Stop the test when the duration ends, the user aborts it, a safety limit is reached, or an error occurs.
- Clean up the worker processes.
- Calculate the summary statistics and save the report.
- Allow the user to view, compare, and export the report.

2.3 Proposed Approach

The proposed system follows a client-server architecture consisting of the host Operating System, the Python backend, and the React frontend. The host Operating System is the source of the actual telemetry and executes the worker processes. The

Python backend acts as the main processing layer: it validates requests, runs the workloads, samples the metrics, and stores the reports. The React frontend acts as the user interface and communicates with the backend through REST API endpoints and a WebSocket stream.

The major modules of the proposed system are:

1. System Monitor

This module uses psutil to collect CPU, per-core, RAM, swap, CPU frequency, uptime, and host metadata.

2. CPU Stress Engine

This module runs CPU-bound work in multiprocessing workers with a configurable intensity.

3. Memory Stress Engine

This module allocates memory in chunks, touches the allocated pages, and checks the available-memory reserve.

4. Test Manager

This module validates the requests, manages the state transitions, coordinates the workers, samples the metrics, and finalizes the tests.

5. Safety Configuration

This module defines the limits for duration, workers, memory reserve, polling interval, and warning thresholds.

6. REST API

This module exposes the health, system, settings, test-control, history, and report endpoints.

7. WebSocket Stream

This module sends the current metric snapshots to the connected clients at a configurable interval.

8. Report Service

This module calculates the summary statistics, persists the JSON reports, lists the history, deletes reports, and generates the CSV export.

9. Frontend Dashboard

This module presents the live telemetry, workload configuration, test progress, history, comparisons, system information, and settings.

2.4 Advantages of the Proposed Approach

The proposed system provides several advantages over manual testing with separate tools.

Centralized Control

Workload configuration, live monitoring, history, and reports are available from a single browser interface.

Controlled and Repeatable Workloads

The test type, duration, worker count, CPU intensity, and memory target can be configured, so that experiments can be repeated with the same settings.

Live Telemetry

The system streams CPU, per-core, memory, and swap measurements to the dashboard, allowing the effect of the workload to be observed while the test is running.

Built-in Safety Limits

Duration limits, worker limits, and a memory reserve reduce the risk of making the computer unresponsive or exhausting the system memory.

Report Persistence and Export

Completed tests are saved as JSON reports and can be exported as JSON or CSV for further analysis.

Educational Value

The project demonstrates practical concepts related to:

- Process management and multiprocessing
- CPU scheduling and yielding
- Memory allocation and demand paging
- Inter-process coordination
- Workload cancellation and process lifecycle
- Resource monitoring and thresholds

- REST APIs and WebSockets
- Client-server architecture

Lightweight and Local

The application runs on a local computer and stores its reports locally, so no remote database or special infrastructure is required.

Expandable Architecture

The modular design allows additional features to be added in the future, such as authentication, multi-machine monitoring, notifications, and PDF reports.

CHAPTER 3 – SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The hardware requirements for the CPU and Memory Stress Testing Tool are modest because the application itself is lightweight. The actual load is created by the workloads that the user configures. The following baseline is recommended:

Component	Recommended Baseline
Processor	Modern multi-core 64-bit CPU
RAM	At least 4 GB; 8 GB or more is preferable for memory tests
Storage	Sufficient free space for Python/Node dependencies and saved reports
Display	Desktop/laptop display suitable for dashboard charts
Input	Keyboard and mouse
Network	Localhost access; Internet may be needed to install dependencies

The actual safe memory target depends on the available RAM of the host. A memory test should not be configured to consume all of the installed memory.

3.2 Software Requirements

The application requires a Python runtime for the backend and a Node.js environment for the frontend. The software and libraries used in the project are listed below:

Software / Library	Purpose
Windows 10/11	Target host environment described by the project README
Python 3.11+ (3.12 verified in README)	Backend runtime
FastAPI	REST API framework
Uvicorn	ASGI server for the backend
psutil	CPU, memory, swap, frequency, uptime, and system information
Pydantic v2	Request/response validation and data schemas
Node.js and npm	Frontend development and package management

Software / Library	Purpose
React 19	Browser interface
Vite	Frontend development server and production build
Recharts	Charts and visualized metrics
pytest / pytest-asyncio / httpx	Backend automated tests
Vitest / React Testing Library	Frontend component tests

3.3 Operating System Environment – Windows 10/11

The supplied implementation is designed for a Windows desktop environment. The Windows host provides the processor, memory, and process-management services that the application measures and stresses. The backend and the frontend run as two separate programs on the same computer and communicate through localhost.

The backend is launched with Uvicorn on port 8000, and the React frontend is served by the Vite development server, typically on port 5173. The application is opened in a modern web browser. Since psutil is used for measurements, the application can obtain real information from the host Operating System rather than displaying simulated data.

Resource measurements depend on the host Operating System and on the permissions that are available to the process.

3.4 Development Tools and Libraries

Python

Python is used to implement the backend logic and the workload engines. Python 3.11 or later is required, and Python 3.12 is verified in the project README.

FastAPI and Uvicorn

FastAPI is used to define the typed REST endpoints and automatically exposes interactive API documentation. Uvicorn runs the FastAPI application as an ASGI server.

psutil

psutil is used to gather host telemetry such as CPU percentage, per-core CPU percentages, memory totals and availability, swap usage, CPU frequency, boot time, and platform information.

Pydantic

Pydantic version 2 is used to validate requests and to define the response schemas of the API.

Node.js and npm

Node.js provides the runtime environment required for the React frontend, and npm is used to install the frontend dependencies and to run the development and build scripts.

React, Vite and Recharts

React is used to develop the single-page interface. Vite provides the development server and the production build. Recharts is used to display the metrics and comparisons as charts.

pytest, Vitest and React Testing Library

pytest, pytest-asyncio, and httpx are used for the backend tests, while Vitest and React Testing Library are used for the frontend component tests.

Browser

A modern web browser is required to access the React-based dashboard.

3.5 Constraints

- Resource measurements depend on the host Operating System and the permissions available to the process.
- CPU frequency information may not be available on every machine; the backend handles this as an optional value.
- Stress testing can affect system responsiveness, battery usage, temperature, and power consumption.
- Reports are stored locally; the supplied project does not use a remote database.
- The supplied CORS configuration permits all origins, which is convenient for local development but should be restricted before deployment beyond a trusted local environment.

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

4.1 Python, FastAPI and Uvicorn

Python is the main programming language used for developing the backend of the application. It implements the telemetry collection, the workload engines, the test manager, and the report service.

FastAPI defines the typed REST endpoints and automatically exposes interactive API documentation. Request and response data are described using Pydantic schemas, which allows invalid input to be rejected before a test is started.

Uvicorn runs the FastAPI application as an ASGI server. The backend application is created in backend/app/main.py, and the documented local launch command uses port 8000.

Therefore, Python with FastAPI and Uvicorn forms the core backend technology of the project.

4.2 psutil

psutil is a cross-platform library used to retrieve information about the running system. In this project it gathers the following host telemetry:

- Total CPU percentage and per-core CPU percentages.
- Total, used, and available memory and the RAM percentage.
- Swap usage.
- CPU frequency, when it is available.
- Boot time and system uptime.
- Platform, processor, and core-count information.

These values are placed in metric snapshots that are sent to the dashboard and stored in the reports.

4.3 Python Multiprocessing

Python multiprocessing is used to create separate worker processes for the CPU and memory workloads. Because each worker is an independent process, CPU-bound work can run in parallel on several logical processors.

Shared values and events allow the manager and the workers to coordinate the status, the cancellation, and the reporting of allocated memory. When the test ends, the workers are stopped and the Operating System reclaims their resources.

4.4 WebSockets and Data Persistence

The backend exposes the /ws/metrics endpoint for streaming the metric snapshots. A frontend hook manages the stream and attempts to reconnect when needed. This allows the dashboard to update continuously without repeated manual requests.

Historical reports are written as JSON files in the data/reports directory. The report service calculates the average and peak CPU and RAM values, records the number of samples, and converts the time-series metrics of a report into CSV for export.

4.5 React, Vite and Recharts

React provides the single-page interface of the application. Vite supports development and production builds, and Recharts is used to display metrics and comparisons.

The frontend includes the Dashboard, Run Test, Live Monitoring, Test History, Reports Comparison, System Information, and Settings pages, with shared navigation and confirmation components.

4.6 Main Project Structure

The repository is organized into backend and frontend folders. The main files and their responsibilities are listed below:

Path	**Responsibility**
backend/app/main.py	FastAPI app, lifecycle, CORS, routers, WebSocket endpoint
backend/app/config.py	Application settings and safety defaults
backend/app/api/system.py	Health, system info, metrics, and settings endpoints
backend/app/api/tests.py	Start, stop, and status endpoints for stress tests
backend/app/api/reports.py	History, report details, exports, and deletion
backend/app/models/schemas.py	Pydantic request and response schemas
backend/app/services/monitor.py	System information and metric snapshots
backend/app/services/cpu_stress.py	CPU workload worker engine

Path	Responsibility
backend/app/services/memory_stress.py	Memory allocation worker engine
backend/app/services/test_manager.py	Test lifecycle, validation, monitoring, and cleanup
backend/app/services/report_service.py	Report calculations, JSON persistence, CSV generation
frontend/src/pages/	Main dashboard and application pages
frontend/src/hooks/useMetricsStream.js	Live metrics WebSocket hook
frontend/src/services/api.js	Frontend REST client
data/reports/	Persisted JSON reports

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application follows a three-layer architecture. The presentation layer is the React frontend. The service/API layer is the FastAPI backend. The host Operating System is the source of the actual telemetry and executes the worker processes. The report service stores the completed test data locally.

The overall architecture can be represented as follows:

User

↓

React Frontend

↓

FastAPI REST API / WebSocket

↓

Monitoring & Test Manager

↓

OS / CPU & Memory Workers

After a test finishes, the data is stored and made available to the user as follows:

Completed Test

↓

Report Service

↓

Local JSON Report

↓

History / JSON / CSV Export

5.2 System Workflow

The system follows a request-driven workflow with continuous monitoring while a test is active.

Step 1: The backend starts, initializes the monitoring service, and ensures that the report directories exist.

Step 2: The frontend requests the system information and the current metrics through

REST endpoints and connects to the metrics WebSocket.

Step 3: The user selects CPU, memory, or combined testing and configures the permitted workload parameters.

Step 4: The backend validates the duration, the worker count, and the memory safety budget before starting the test.

Step 5: The test manager starts the required workload engine(s) and periodically samples the system metrics.

Step 6: The dashboard updates its cards, charts, test progress, and live monitoring views.

Step 7: The test ends when the configured duration is reached, the user stops it, a safety limit is activated, or an error occurs.

Step 8: The backend cleans up the worker processes, calculates the summary statistics, and saves the completed report.

Step 9: The user can review the history, inspect a detailed report, compare runs, or export the report data.

5.3 Flowchart / Data Flow

The data flow of the system begins with the host Operating System. The monitoring service reads the resource measurements, and the test manager adds the test status, elapsed time, active workers, and allocated memory to each snapshot. The snapshots are then streamed to the React frontend through the WebSocket endpoint and are also collected as a time series during a test.

When the test is finalized, the collected samples are used by the report service to calculate the summary statistics. The complete report is stored in a local JSON file and can later be listed in the history or exported.

Data Flow:

Host Operating System

↓

System Monitor (psutil)

↓

Test Manager (status, elapsed time, workers)

↓

WebSocket / REST API

↓

React Dashboard

↓

Report Service → JSON Report → JSON / CSV Export

5.4 Module Description

The CPU and Memory Stress Testing Tool consists of several modules.

System Monitor

Collects CPU, per-core, RAM, swap, CPU frequency, uptime, and host metadata.

CPU Stress Engine

Runs CPU-bound work in multiprocessing workers with configurable intensity.

Memory Stress Engine

Allocates memory in chunks, touches the allocated pages, and checks the available-memory reserve.

Test Manager

Validates requests, manages state transitions, coordinates workers, samples metrics, and finalizes tests.

Safety Configuration

Defines the duration, worker, memory reserve, polling, and warning-threshold limits.

REST API

Exposes the health, system, settings, test-control, history, and report endpoints.

WebSocket Stream

Sends the current metric snapshots to the connected clients at a configurable interval.

Report Service

Calculates the summary statistics, persists the JSON reports, lists the history, deletes reports, and exports CSV.

Frontend Dashboard

Presents the live telemetry, workload configuration, test progress, history, comparisons, system information, and settings.

5.5 Input and Output Design

Input Design

The inputs to the system are the test configuration and the user actions. The inputs include:

- Test name and test type: CPU, memory, or combined.
- Duration and optional warm-up time.
- CPU worker count and intensity.
- Target memory allocation in MB.
- Monitoring interval and safety settings.
- User action to start or stop a test, inspect history, compare reports, or export results.

Output Design

The system provides the following outputs:

- Current CPU percentage and per-core utilization.
- RAM and swap totals, usage, available memory, and percentages.
- CPU frequency when available, uptime, and system metadata.
- Test status, elapsed time, progress, active workers, and allocated memory.
- Average and peak CPU/RAM metrics, sample count, test status, and termination reason.
- Saved report history and downloadable JSON/CSV exports.

The outputs are displayed through the React-based dashboard in an organized and user-friendly format.

CHAPTER 6 – IMPLEMENTATION

6.1 Module-wise Implementation

The CPU and Memory Stress Testing Tool is implemented using a Python backend and a React frontend. The backend generates the workloads, collects the telemetry, controls the test lifecycle, and stores the reports. The frontend presents the live information and the saved results to the user.

System Monitoring

The SystemMonitor service uses psutil to create snapshots. Each snapshot includes a timestamp, total CPU percentage, per-core percentages, total/used/available RAM, RAM percentage, swap values, CPU frequency when available, uptime, active worker count, allocated memory, current test status, test identifier, and elapsed time. The system information includes the operating-system details, Python version, CPU architecture, processor description, physical and logical cores, and total RAM.

CPU Stress Engine

The CPU engine launches independent worker processes. The workers execute computational work for a portion of a duty-cycle window and sleep for the remaining portion according to the requested intensity. This design supports controlled changes in CPU demand. The worker count is validated against the configured maximum before the test begins.

Memory Stress Engine

The memory engine allocates memory in bounded chunks rather than requesting the entire target as one allocation. It writes data into the allocated buffers to touch the memory pages and make the workload visible to the Operating System. The engine shares its current allocated-memory value with the manager and signals when the safety reserve is reached. On stop or safety abort, the engine attempts to release the resources and terminate its worker.

Test Lifecycle and Safety

The TestManager validates a request before starting it. It rejects a new request while another test is starting, running, or stopping. It tracks the IDLE, STARTING, RUNNING, STOPPING, COMPLETED, STOPPED, and FAILED states, and supports

manual cancellation, configured-duration completion, failure handling, and cleanup during application shutdown.

Reports and Exports

At test finalization the report service calculates the summary statistics and stores the complete report as a JSON file. The report can later be listed, retrieved, deleted, or exported as JSON or CSV.

6.2 Algorithm / Pseudocode

CPU Stress Algorithm

The CPU stress algorithm generates a controlled CPU load using a duty cycle.

Step 1: Receive the worker count and the CPU intensity from the validated test request.

Step 2: Create the requested number of worker processes.

Step 3: In each worker, define a duty-cycle window and compute the busy portion from the intensity.

Step 4: Perform computational work during the busy portion of the window.

Step 5: Sleep for the remaining portion of the window so that the scheduler can run other processes.

Step 6: Check the stop event at the end of every cycle.

Step 7: Repeat steps 4 to 6 until the stop event is set.

Step 8: Exit the worker so that the Operating System reclaims its resources.

Memory Stress Algorithm

The memory stress algorithm increases the memory usage gradually while protecting the system.

Step 1: Receive the target memory size in MB from the validated test request.

Step 2: Allocate a bounded chunk of memory.

Step 3: Write data into the chunk so that its pages are touched and committed.

Step 4: Update the shared allocated-memory value for the test manager.

Step 5: Check the available memory. If it has reached the safety reserve, signal the manager and stop allocating.

Step 6: Repeat steps 2 to 5 until the target is reached or the reserve is reached.

Step 7: Hold the allocated memory until the stop event is set or the duration ends.

Step 8: Release the buffers and terminate the worker.

Test Start and Validation Algorithm

Step 1: The user submits the test type and the configuration through the start endpoint.

Step 2: The manager rejects the request if another test is starting, running, or stopping.

Step 3: The manager checks that the duration is at least five seconds and does not exceed the maximum setting.

Step 4: For CPU and combined tests, the worker count is checked against the worker limit.

Step 5: For memory and combined tests, the safe memory budget is calculated by subtracting the larger of the fixed reserve and the percentage of total RAM from the currently available RAM.

Step 6: If the requested memory exceeds the safe budget, the request is rejected.

Step 7: The state changes to STARTING, the workers are created, and the state changes to RUNNING.

Test Stop and Finalization Algorithm

Step 1: The test ends when the duration is reached, the user requests a stop, a safety limit is reached, or an error occurs.

Step 2: The state changes to STOPPING and the stop event is set for the workers.

Step 3: The manager waits for the workers to exit and terminates them if necessary.

Step 4: The summary statistics are calculated from the collected samples.

Step 5: The report is saved as a JSON file.

Step 6: The final state is set to COMPLETED, STOPPED, or FAILED, and the system returns to the idle condition.

6.3 Core Python Implementation

The backend of the application is implemented in Python using FastAPI. The API routes, schemas, monitoring service, stress engines, test manager, and report service are separated into different modules so that each part has a clear responsibility.

The TestManager is the central component. It validates the requests, starts the workload engine(s), samples the metrics at the configured interval, and finalizes the test. The

safety defaults used by the supplied code are listed below:

Safety Setting	Default	Purpose
Maximum test duration	300 seconds	Limits how long one test can run
Default test duration	30 seconds	Initial duration value
Maximum CPU workers	16	Restricts the CPU worker count
Minimum available RAM reserve	1024 MB	Keeps a minimum memory reserve
RAM reserve percentage	15%	Reserves a share of total RAM
Default monitoring interval	1.0 second	Controls the sampling/streaming cadence
CPU warning threshold	90%	Default CPU warning setting
RAM warning threshold	85%	Default RAM warning setting

6.4 System Calls / APIs Used

The system uses Operating System services through Python libraries rather than through direct system calls.

- **psutil:** used to read CPU, per-core, memory, swap, frequency, boot time, and platform information from the host.
- **Python multiprocessing:** used to create worker processes and to use events and shared values for coordination.
- **REST API (FastAPI):** used for system information, settings, test control, history, and report endpoints.
- **WebSocket (/ws/metrics):** used to stream the live metric snapshots to the dashboard.
- **File system:** used to store reports as JSON files in the data/reports directory.

Interactive API documentation is available at http://127.0.0.1:8000/docs after the backend starts.

6.5 Process / Thread Implementation

The CPU workload and the memory workload are executed in separate worker processes created with Python's multiprocessing facilities. Using processes instead of threads allows CPU-bound work to run in parallel on different logical processors, and

each worker has its own address space.

The test manager runs in the backend application and supervises these workers. It creates them when a test starts, observes their status, and stops them when the test ends. During application shutdown, the manager also performs cleanup so that no worker process is left running.

6.6 IPC / Synchronization Implementation

Communication between the manager and the workers is implemented using multiprocessing coordination primitives. An **event** is used to signal a stop request to the workers, and **shared values** are used to report the status and the amount of allocated memory back to the manager.

Communication between the React frontend and the backend is implemented using HTTP REST requests for commands and queries, and a WebSocket connection for the continuous metric stream. The frontend hook reconnects when the connection is interrupted.

The manager also prevents a new test from starting while another test is starting, running, or stopping, so that two tests never compete for the same resources.

6.7 Memory Management Implementation

Memory behaviour is implemented by allocating buffers in bounded chunks and writing data into them so that the pages are actually touched. This makes the workload visible as committed physical memory rather than only reserved virtual address space.

Before a memory or combined test starts, the manager calculates a safe memory budget. The budget is the currently available RAM minus the larger of a fixed reserve (1024 MB by default) and a percentage of the total RAM (15% by default). A request that exceeds this budget is rejected. During the test, the engine continues to monitor the available memory and stops further allocation when the reserve threshold is reached.

6.8 User Interface Implementation

The user interface is developed using React, with Vite for the build and Recharts for the charts. It contains the following pages and components:

Page / Component	**Purpose**

Page / Component	Purpose
Dashboard	Overview cards and charts for current CPU and memory metrics.
Run Test	Configure the workload type, duration, CPU workers/intensity, and memory target.
Live Monitoring	Detailed live telemetry including per-core gauges and memory/swap breakdown.
Test History	Browse, filter, sort, and inspect saved test runs; access exports.
Reports Comparison	Compare selected runs using charts and a comparison matrix.
System Information	Display host hardware and operating-system information.
Settings	View and update safety limits and warning thresholds.
Active Test Banner	Shows active test progress and provides an abort control.
Confirmation Modal	Confirms sensitive or destructive actions before proceeding.

6.9 Report Generation and Persistence

At test finalization, the report service calculates the average and peak CPU utilization, the average and peak RAM utilization, the peak allocated memory, the actual duration, the sample count, and the configured workload parameters. The complete report includes the test identifiers, timestamps, status, termination reason, configuration, system information, summary statistics, and the time-series metrics.

Reports are stored in data/reports as JSON. The report service validates the report identifiers and checks the resolved paths to reduce the risk of path traversal.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

7.1 Installation and Setup

The CPU and Memory Stress Testing Tool is developed for Windows and is executed using two terminals: one for the backend and one for the frontend. The backend directory is opened first and a Python virtual environment is created:

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

The frontend dependencies are installed separately in the frontend directory using npm.

7.2 Execution Commands

After the installation, the backend is started using Uvicorn:

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

The frontend is started in a second terminal:

```
cd frontend
npm install
npm run dev
```

The Vite development server prints a local address, typically http://127.0.0.1:5173. This address is opened in a browser to use the dashboard. The backend terminal and the frontend terminal must be kept running while the application is used. Stress tests should be run conservatively and the host computer should be monitored.

7.3 Sample Inputs and Outputs

The system accepts the test configuration from the user through the dashboard and returns the live measurements and the final report.

Sample Input 1: CPU-only Test

The user selects the CPU test type, a short duration, a modest worker count, and a CPU intensity.

Sample Output: The dashboard shows the CPU percentage and the per-core utilization increasing, the active test banner shows the progress, and a report is saved when the duration ends.

Sample Input 2: Memory-only Test

The user selects the memory test type and a conservative memory target that leaves adequate memory available.

Sample Output: The allocated memory grows in chunks up to the target or until the safety reserve is reached, and the RAM percentage on the dashboard increases accordingly.

Sample Input 3: Manual Stop

The user starts a test and presses the abort control in the active test banner.

Sample Output: The backend stops the workers, the state changes to STOPPED, and the report records the termination reason.

7.4 API Reference and Testing Strategy

The backend exposes the following REST and WebSocket endpoints:

Method	Endpoint	Purpose
GET	/api/health	Check backend status and version.
GET	/api/system/info	Retrieve host system information.
GET	/api/system/metrics	Retrieve the latest metric snapshot.
GET	/api/settings	Read current safety settings.
POST	/api/settings	Update supported safety settings within schema bounds.
POST	/api/settings/reset	Restore default safety settings.
POST	/api/tests/start	Validate and start a stress test.
POST	/api/tests/{test_id}/stop	Stop a test using its identifier.
POST	/api/tests/stop	Stop the active test.
GET	/api/tests/status	Get current test state and progress.
GET	/api/tests/history	List saved test summaries.
GET	/api/tests/{test_id}/report	Retrieve a detailed report.
GET	/api/tests/{test_id}/export/json	Download report as JSON.

Method	Endpoint	Purpose
GET	/api/tests/{test_id}/export/csv	Download time-series metrics as CSV.
DELETE	/api/tests/{test_id}	Delete one report.
DELETE	/api/tests/	Clear saved report history.
WebSocket	/ws/metrics	Stream live metric snapshots.

The repository includes backend pytest tests for the API endpoints, report handling, stress engines, and system monitoring. The project README describes 12 backend tests and 4 frontend tests. The frontend uses Vitest and React Testing Library to test key interface behaviours, and the production frontend bundle can be checked with npm run build.

Test Area	Example Validation
Health and system APIs	Confirm the health response and system information schema.
Workload validation	Reject invalid durations, excessive worker counts, and unsafe memory targets.
Start/stop lifecycle	Start a test, inspect status, request cancellation, and confirm cleanup.
Automatic completion	Confirm a test reaches a terminal state after its configured duration.
Memory safety	Check that the reserve is respected and unsafe allocation is rejected or stopped.
Report persistence	Confirm completed runs are saved and listed in history.
Export	Verify JSON report and CSV metric export endpoints.
Frontend	Verify dashboard metrics, form validation, and abort controls.
Production build	Run the Vite build to detect frontend compilation errors.

7.5 Screenshots of Execution

The observed CPU and RAM values depend on the machine used for the demonstration. Therefore, the screenshots below should be captured after running the application on the target computer.

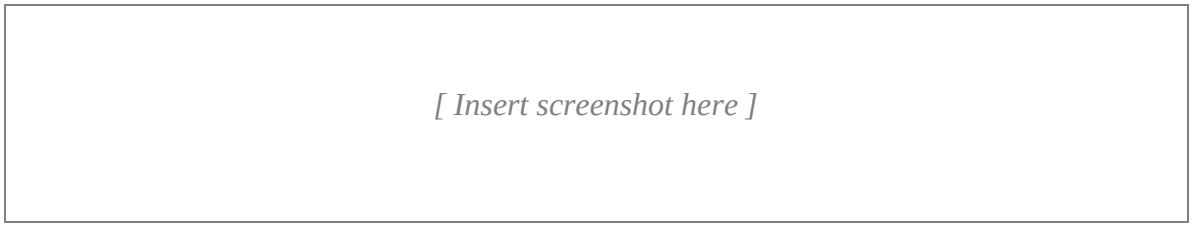


Figure 7.1: Backend running with Uvicorn (terminal output)

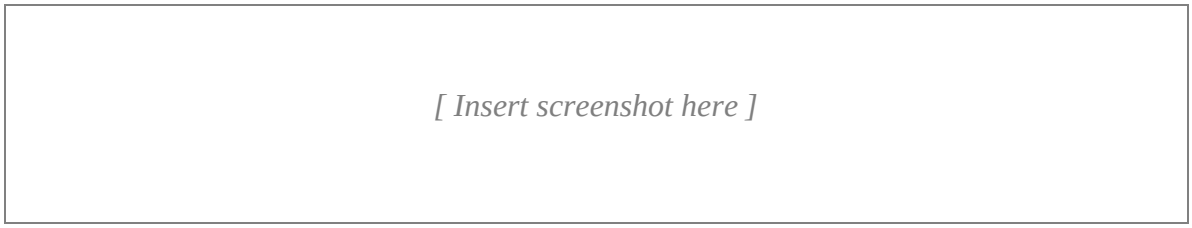


Figure 7.2: Frontend running with Vite (terminal output)

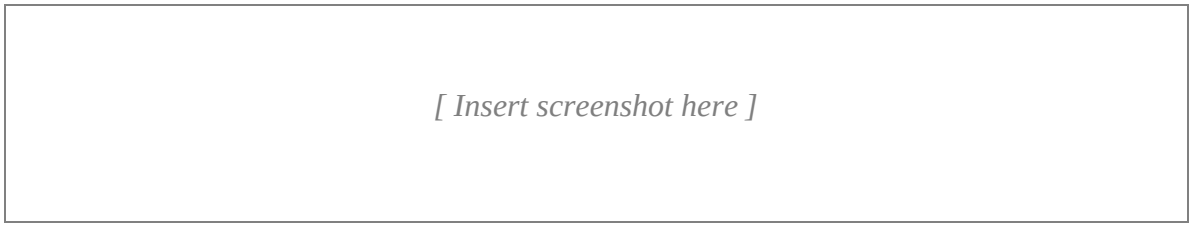


Figure 7.3: Dashboard with live CPU and memory cards

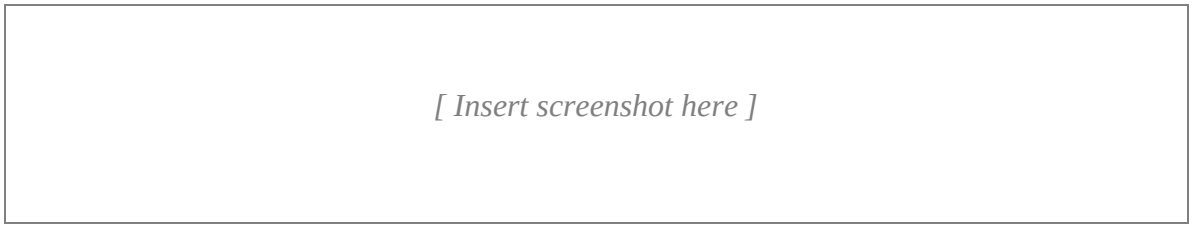


Figure 7.4: Run Test page with a CPU-only configuration

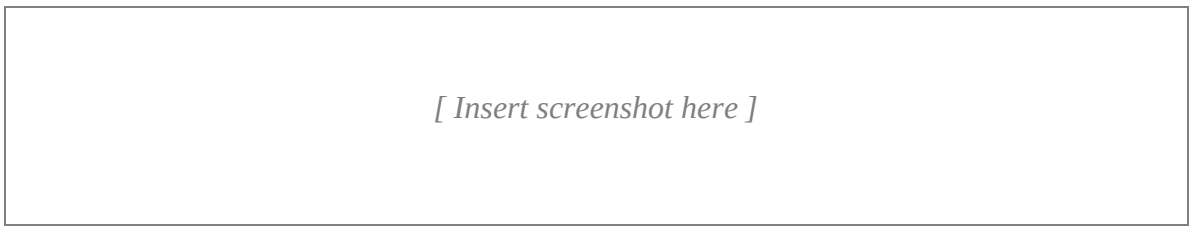


Figure 7.5: Live Monitoring page showing per-core utilization

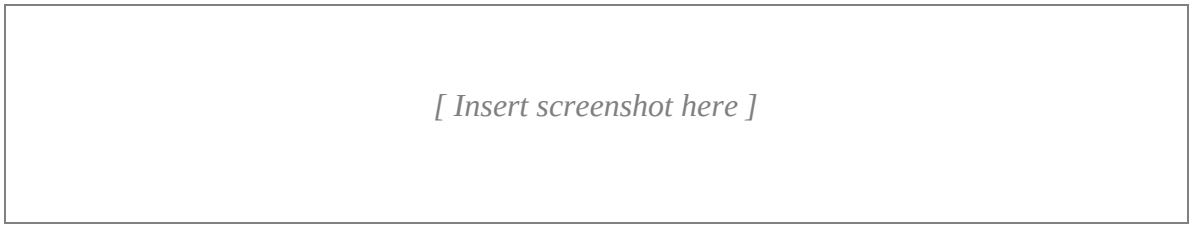


Figure 7.6: Test History and detailed report view

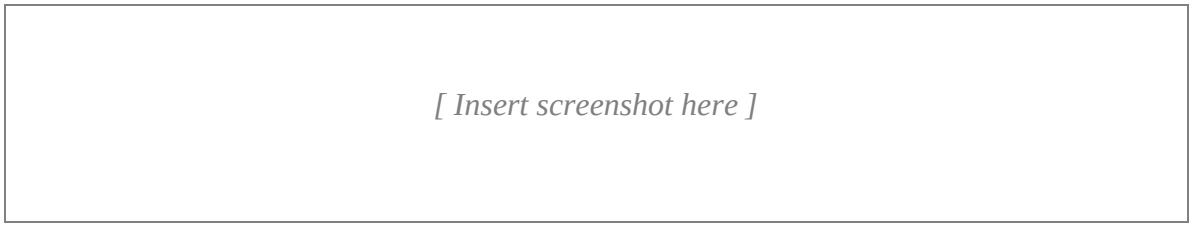


Figure 7.7: Reports Comparison page

7.6 Demonstration Procedure

Step 1: Start the backend and the frontend using the commands given above.

Step 2: Open the dashboard and identify the live CPU, memory, and system information cards.

Step 3: Open Live Monitoring and change the telemetry interval if the interface offers that control.

Step 4: Run a short CPU-only test with a modest worker count and observe the CPU usage in the dashboard and in Windows Task Manager.

Step 5: Run a small memory-only test with a conservative target, ensuring that adequate memory remains available.

Step 6: Run a combined test only if the host has sufficient resources.

Step 7: Stop a test manually and observe the terminal status and the report.

Step 8: Open Test History, inspect the detailed report, compare runs, and export JSON or CSV.

Step 9: Restore the default safety settings after experimentation if the settings were changed.

7.7 Demonstration of OS Concepts

The project demonstrates several important Operating System concepts through practical implementation.

Process Management:

The CPU and memory workloads run as separate worker processes, and the test manager creates, supervises, and terminates them.

CPU Scheduling:

The duty-cycle pattern of busy work and sleeping shows how the scheduler shares processor time and how the configured intensity affects CPU utilization.

Memory Management:

Memory is allocated in chunks and touched so that pages are committed to the working set, and the available memory is monitored against a safety reserve.

Demand Paging and Virtual Memory:

The difference between reserved address space and committed physical memory is observed through the RAM and swap measurements.

Inter-Process Communication:

Events and shared values are used for stop requests and for reporting allocated memory and worker status.

Process Lifecycle:

The test states IDLE, STARTING, RUNNING, STOPPING, COMPLETED, STOPPED, and FAILED represent the lifecycle of a workload, including cancellation and cleanup.

Resource Monitoring:

CPU, per-core, memory, swap, frequency, and uptime values are collected as time-series data and compared with warning thresholds.

Reporting:

The measurements of each run are stored and can be exported for further analysis.

Therefore, the project provides a practical demonstration of Operating System concepts through a real-time performance testing application.

CHAPTER 8 – CONCLUSION

8.1 Summary of the Project

The CPU and Memory Stress Testing Tool provides a structured way to create controlled workloads and observe system performance through a browser. Its Python backend coordinates multiprocessing-based CPU and memory engines, uses psutil for telemetry, exposes REST and WebSocket interfaces, and stores the test results as JSON with CSV export support.

The React interface organizes workload configuration, live monitoring, history, comparison, system information, and settings in one application. The safety limits and the report history make short academic experiments more controlled and repeatable.

The project demonstrates important Operating Systems concepts in a practical setting: processes and parallel execution, CPU scheduling and yielding, memory allocation and page commitment, resource thresholds, cancellation, process lifecycle management, and performance measurement.

8.2 Objectives Achieved

The major objectives of the project have been achieved.

- Developed a web-based CPU and memory stress testing application.
- Generated CPU-only, memory-only, and combined workloads.
- Monitored total and per-core CPU utilization, memory, and swap.
- Allowed configuration of duration, worker count, CPU intensity, and memory target within limits.
- Streamed live telemetry through a WebSocket endpoint and displayed it using charts.
- Supported manual cancellation and automatic completion.
- Preserved a memory safety reserve to avoid exhausting the system memory.
- Saved reports locally and supported JSON and CSV export.
- Demonstrated Operating System concepts through practical implementation.

8.3 Results Achieved

The developed tool successfully generates controlled workloads and displays the resulting system behaviour through the web dashboard. The system information and the live metrics are retrieved from the host and presented through cards and charts.

The safety mechanisms validate the test duration, the worker count, and the memory budget before a test is started, and the manager stops memory allocation when the reserve is reached. A test can be stopped manually or ends automatically after the configured duration, and the worker processes are cleaned up.

Completed tests are saved as reports, which can be browsed in the history, compared, and exported as JSON or CSV. The automated tests cover the main backend behaviours and key frontend behaviours. Numerical performance results depend on the machine used and should be recorded after running the application on the target computer.

8.4 Skills and Learning Outcomes

The project helped in developing both technical and practical skills related to Operating Systems and web application development. The major learning outcomes include:

- Understanding processes, multiprocessing, and process lifecycle management.
- Understanding CPU scheduling and the effect of workload intensity.
- Understanding memory allocation, demand paging, and memory reserves.
- Working with psutil for system telemetry.
- Developing a backend application using Python and FastAPI.
- Creating and using REST API endpoints and WebSockets.
- Developing a dashboard using React, Vite, and Recharts.
- Using events and shared values for inter-process coordination.
- Writing automated tests with pytest and Vitest.
- Integrating Operating System concepts into a real-world application.

8.5 Limitations

- The supplied design is intended for local desktop-browser use and stores reports locally rather than in a shared database.
- Results vary with hardware, background applications, power mode, thermal

limits, and Operating System behaviour.

- Stress workloads may temporarily reduce system responsiveness; conservative settings are important.
- Authentication and role-based access control are not part of the supplied implementation.
- The current CORS policy is permissive and should be tightened before any network-facing deployment.

8.6 Future Enhancements

- Add user authentication and authorization for test controls and report deletion.
- Provide configurable report retention, backup, and import functionality.
- Support multiple host agents and centralized comparison across machines.
- Add notifications when configurable thresholds are crossed.
- Include downloadable PDF reports and richer experiment annotations.
- Add stronger audit logs and deployment-specific CORS/network restrictions.
- Improve accessibility, responsive layouts, and additional automated end-to-end tests.

REFERENCES

9.1 Books

- Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, Wiley.
- Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, Pearson.

9.2 Research Papers

Research papers and academic materials related to CPU scheduling, memory management, stress testing, system resource monitoring, and operating system performance measurement were referred to during the development of the project.

These resources helped in understanding workload generation, resource utilization, process lifecycle management, and performance reporting.

9.3 Python and Library Documentation

The following documentation resources were referred to:

- FastAPI Documentation. https://fastapi.tiangolo.com/
- psutil Documentation. https://psutil.readthedocs.io/
- Python Documentation – multiprocessing. https://docs.python.org/3/library/multiprocessing.html
- Pydantic Documentation. https://docs.pydantic.dev/
- pytest Documentation. https://docs.pytest.org/

9.4 Online Resources

The following online documentation resources were referred to during implementation:

- React Documentation – React components and state management. https://react.dev/
- Vite Documentation – frontend development server and build. https://vite.dev/
- Recharts Documentation – charts and visualized metrics. https://recharts.org/
- Vitest Documentation – frontend testing. https://vitest.dev/
- Project source archive: CPU and Memory Stress Testing Tool (backend,

frontend, README, tests, and sample report data).

These references supported the development of the Python backend, the React frontend, the telemetry collection, the REST API and WebSocket communication, and the Operating System features used in the project.

APPENDIX A – SAMPLE TEST CONFIGURATION CHECKLIST

- Confirm the machine is not running critical or unsaved work.
- Check the current available memory before configuring a memory test.
- Use a short duration for the first run.
- Choose a CPU worker count appropriate to the logical core count of the host.
- Keep the memory reserve at its default unless there is a clear reason to change it.
- Observe the temperature and system responsiveness during the test.
- Stop the test immediately if the system becomes unstable.
- Save or export the report after the test finishes.

APPENDIX B – REPORT FIELDS

A saved report includes the test identifier and name, test type, final status, termination reason, start and end timestamps, test configuration, host system information, summary statistics, and the sequence of metric snapshots.

The summary statistics include the average and peak CPU percentage, the average and peak RAM percentage, the peak allocated memory, the actual duration, the sample count, the CPU worker count, the CPU intensity, and the configured memory target where applicable.